CH13
實務上的最佳實踐

主講：邱偉豪

1 超參數優化
(Hyperparameter tuning)

2 模型集成
(Model ensembling)

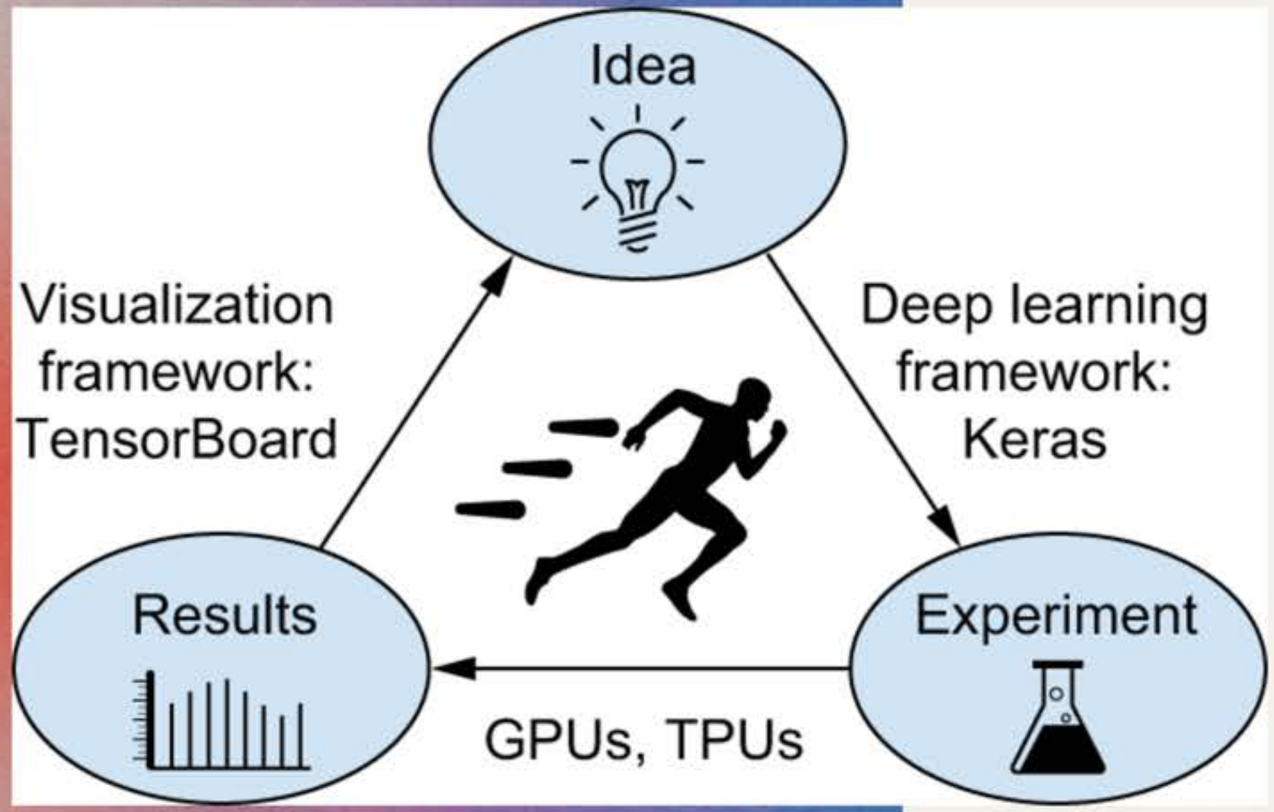
3 分散式訓練
(Multi-GPU & TPU)

4 混合精度與量化
(Mixed-precision & Quantization)

相關檔案

- Google Colab
- DL with Python

深度學習的循環：想法、實驗與結果



深度學習的開發流程

- 深度學習開發是一個不斷迭代的循環：
  - Idea -> Experiment -> Results
- 成功的關鍵在於縮短這個循環的時間
- 我們需要更好的工具（KerasTuner、TensorBoard）與更強的硬體（GPU、TPU）來加速實驗驗證的過程

進展循環(Loop of Progress)

- 圖片展示的是開發流程，中間奔跑的人代表開發者
- 透過Keras實作想法 (Idea)
- 利用GPU/TPU進行實驗(Experiment)
- 最後由 TensorBoard檢視結果 (Results)，形成一個快速回饋的閉環

超參數優化 (Hyperparameter Tuning)

拒絕手動調參：使用 KerasTuner

- 模型的性能高度依賴超參數（如層數、單元數、學習率），但手動調整如同大海撈針
- KerasTuner 是一個自動化搜尋框架，能幫助我們找到最佳參數組合
- 貝氏優化 (Bayesian Optimization)：比隨機搜尋更聰明，它會根據過去的實驗結果來預測下一個可能的最佳參數。

```python
import keras_tuner as kt

def build_model(hp):
    # 定義搜尋空間：單元數在 16~64 之間
    units = hp.Int("units", min_value=16, max_value=64, step=16)
    model = keras.Sequential([
        layers.Dense(units, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])
    # 定義搜尋空間：優化器選擇
    optimizer = hp.Choice("optimizer", ["rmsprop", "adam"])
    model.compile(optimizer=optimizer, loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model

# 使用貝氏優化開始搜尋
tuner = kt.BayesianOptimization(build_model, objective="val_accuracy", max_trials=100)
```

使用 Keras Tuner 進行超參數優化

- 定義搜尋空間 (Search Space)：將固定參數改為變數範圍（如 hp.Int 或 hp.Choice），定義實驗邊界
- 貝氏優化預測：非隨機盲測，而是根據先前結果自動推導勝率(val_accuracy)最高的參數組合
- 自動迭代：執行100次自動實驗(max_trials=100)，取代人工手動測試，快速收斂至最優模型

藉由N-gram或是每個單字出現的次數，在表示法上再新增一些資訊：
{ the:2 , the cat:1 , cat: 1, cat sat: 1, sat: 1, sat on: 1, on: 1, on the: 1, the mat: 1, mat: 1 }

模型集成
(Model Ensembling)

模型集成：群眾的智慧

- 概念：訓練多個不同的模型，並將它們的預測結果取平均
- 優勢：通常能額外提升 1~2% 的準確率，是競賽獲勝的關鍵技巧
- 關鍵：
  - 模型之間必須具備多樣性 (Diversity)
  - 如果模型都犯一樣的錯，集成便無效
  - 可透過不同架構或不同初始化種子來達成

```
# 假設有三個訓練好的模型：model_a, model_b, model_c
preds_a = model_a.predict(x_val)
preds_b = model_b.predict(x_val)
preds_c = model_c.predict(x_val)

# 簡單平均集成
final_preds = 0.33 * preds_a + 0.33 * preds_b + 0.33 * preds_c
```

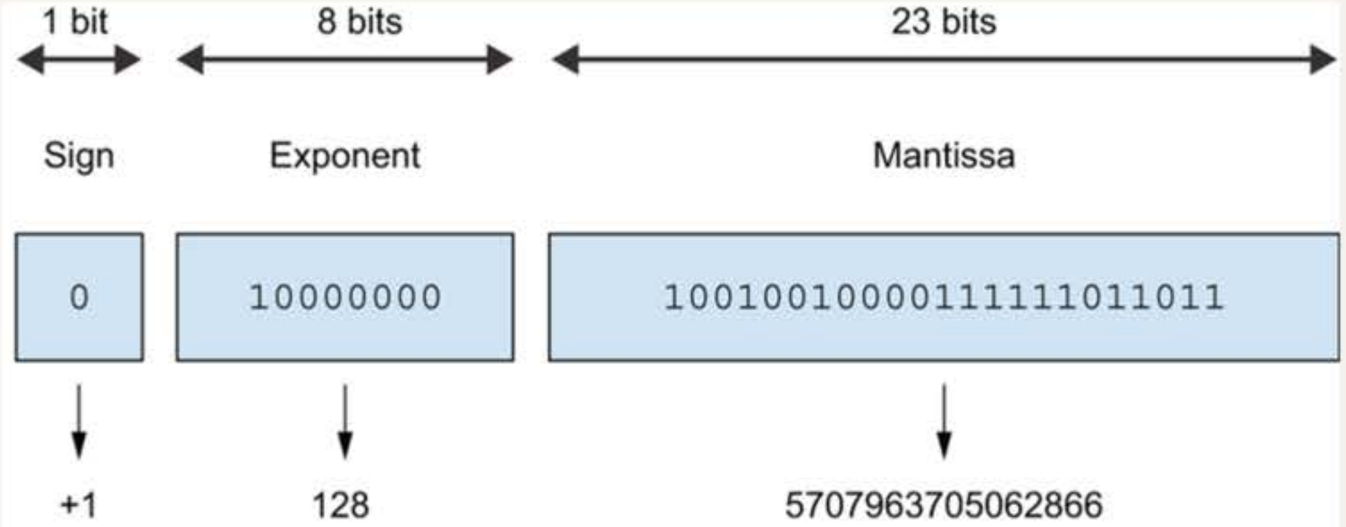
模型集成(Model Ensemble)的核心邏輯

- 個別預測：讓三個不同的模型對同一組資料進行判斷。
- 加權總和：透過 0.33 的權重將結果相加，這本質上是在取平均值。
- 為什麼有效?
  - 降低風險：單一模型可能在特定樣本上看走眼，但三個模型同時出錯的機率較低。
  - 截長補短：只要模型具備多樣性（例如 A 擅長辨識貓、B 擅長辨識狗），集成後的結果會比單一模型更精準

混合精度訓練 (Mixed-Precision)

混合精度：釋放 GPU 的極致效能

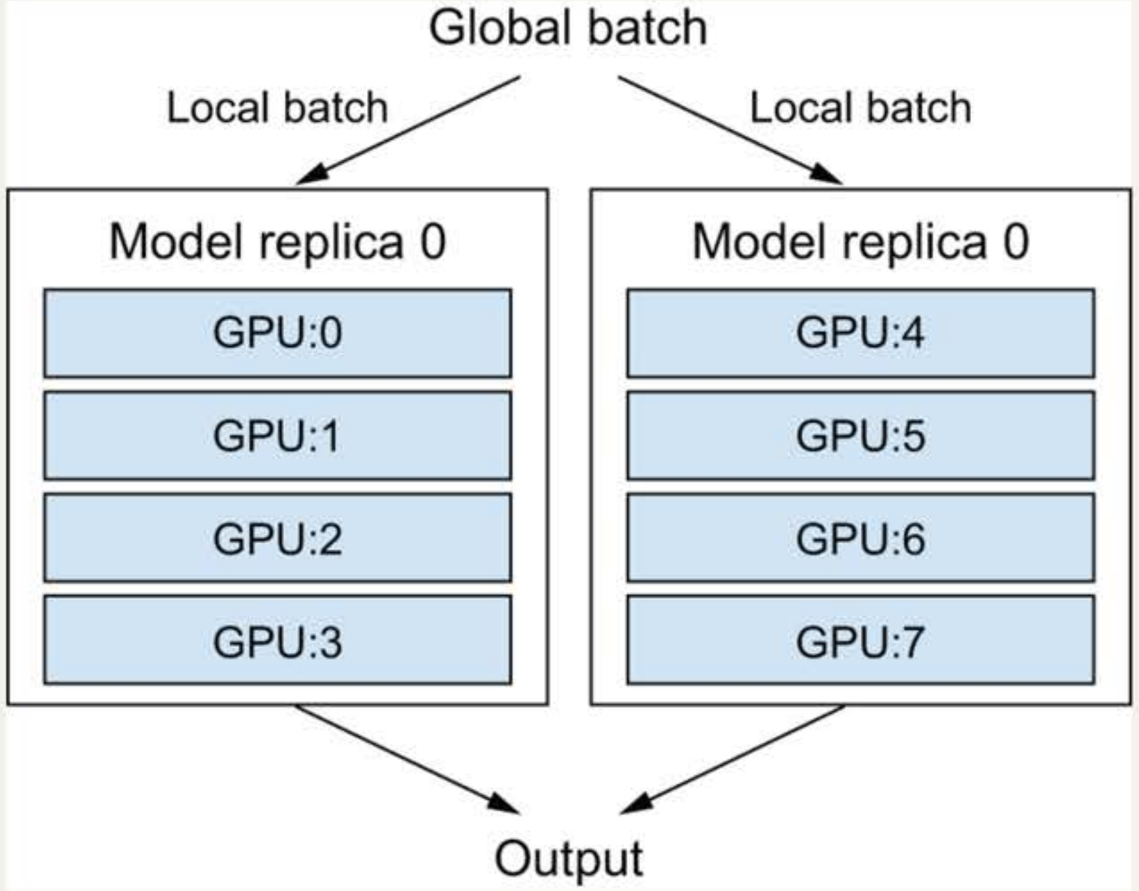
- 原理：
  - 傳統訓練使用 32 位元浮點數 (float32)
  - 混合精度則在運算時使用 16 位元 (float16)，僅在儲存變數時維持 32 位元
- 效益：
  - 利用 NVIDIA GPU 的 Tensor Cores 加速矩陣運算
  - 顯存佔用減半，允許更大的 Batch Size




```
value = +1 * (2 ** (128 - 127)) * 1.5707963705062866
value = 3.1415927410125732
```

- 浮點數的位元結構（符號、指數、尾數）
- 混合精度即是透過減少這些位元數（從 32 bits 降至 16 bits）
- 在不犧牲準確率的前提下換取倍數級的效能提升

多 GPU 分散式訓練 (Data Parallelism)



在多個 GPU 上訓練

- 當單張 GPU 顯存不足或訓練太慢時，使用 MirroredStrategy
- 運作方式：
  - 複製模型：將模型複製到所有 GPU 上
  - 切分數據：將 Global Batch 拆分為多個 Local Batch 分發給各 GPU
  - 同步更新：合併各 GPU 計算出的梯度，同步更新權重

圖片展示Global batch如何被拆解並分發到不同的Model replica (模型副本)與 GPU (0-7) 上進行平行運算，最後再匯總輸出

在 TPU 上進行訓練

TPU：Google 的深度學習超級引擎

- TPU (Tensor Processing Unit)：專為深度學習矩陣運算設計的專用晶片
- 比 GPU 更快、更節能，特別適合處理超大規模數據
- 在 Colab 中使用 TPU 非常簡單，程式碼結構與多 GPU 訓練類似

```python
import tensorflow as tf
# 連接 TPU 叢集
tpu = tf.distribute.cluster_resolver.TPUClusterResolver.connect()

# 定義 TPU 策略
strategy = tf.distribute.TPUStrategy(tpu)

with strategy.scope():
    model = build_model()
    model.compile(...)
```

只需透過 TPUStrategy，即可將原本在 CPU/GPU 跑的程式碼無縫遷移到 TPU 上

從實驗室到生產環境

- 追求效能 (Maximize Performance)：
  - 使用 KerasTuner 自動尋找最佳超參數
  - 使用 模型集成 (Ensembling) 榨出最後一點準確率
- 追求速度 (Scale Up)：
  - 開啟 混合精度 (Mixed-Precision)，一行程式碼讓速度翻倍
  - 利用 Multi-GPU 或 TPU 進行分散式訓練，大幅縮短迭代週期

CH14
結語

主講：邱偉豪

1 回顧關鍵概念

2 深度學習的侷限性

3 為提高 AI 普適性設定方向

4 深度學習的未來

相關檔案

- Google Colab
- DL with Python

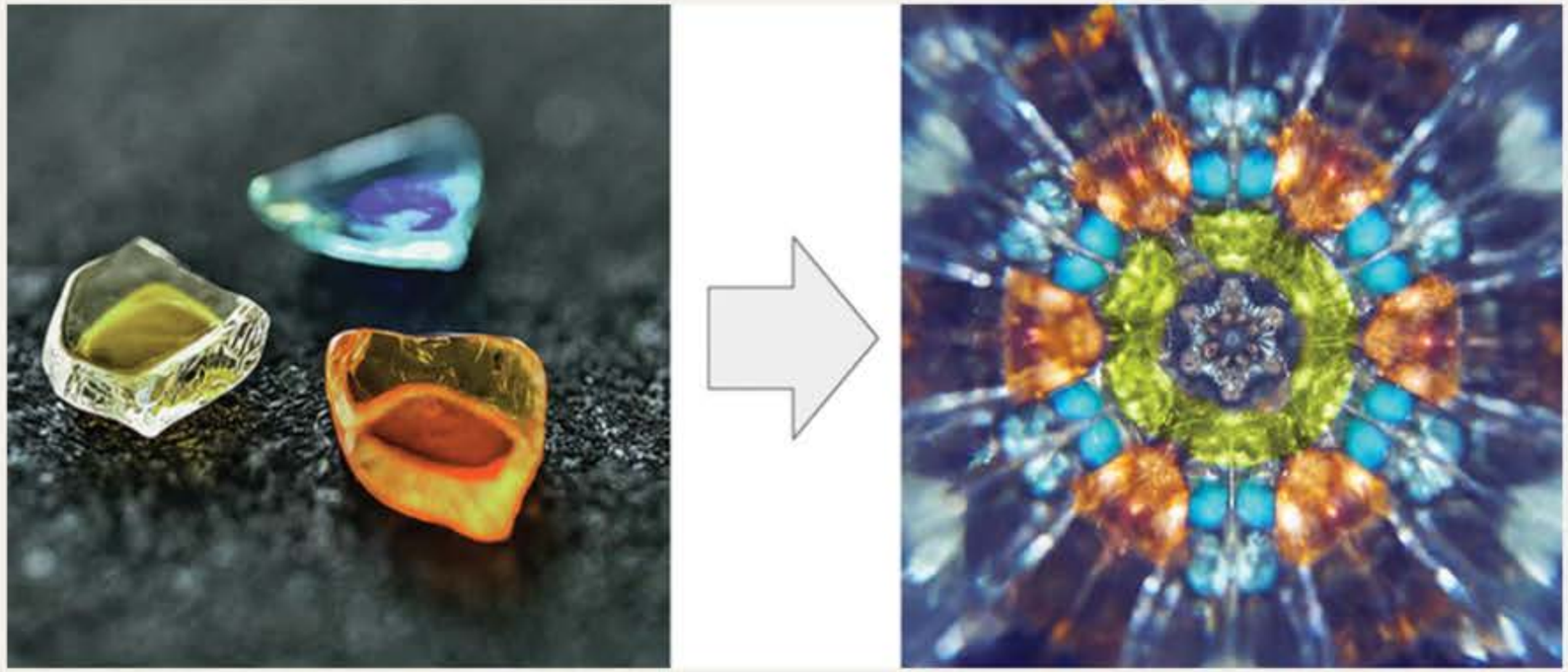
回顧關鍵概念

核心概念

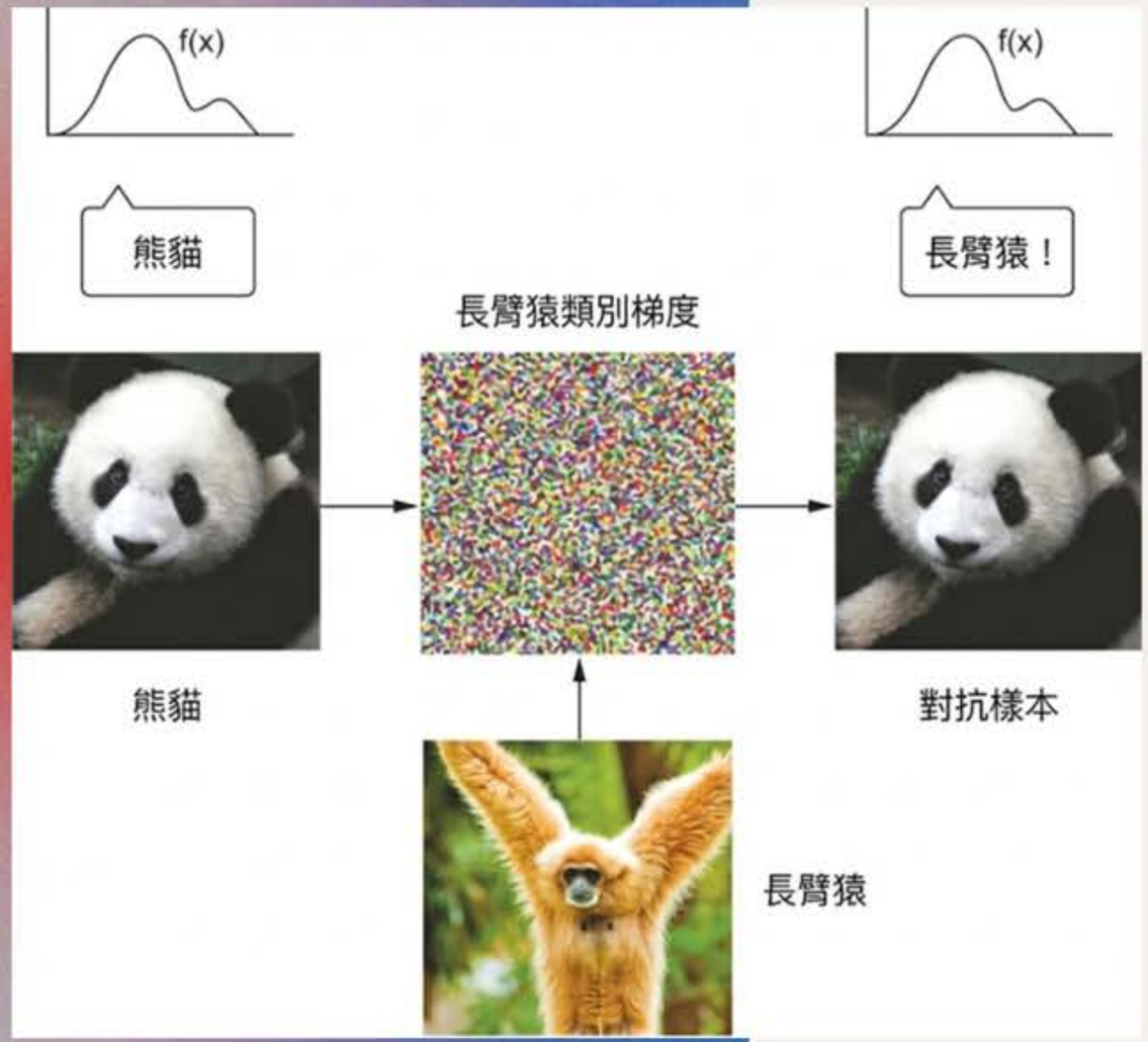
- 深度學習就像是「超級萬花筒」。
- 它把混亂的數據放進去，透過一層層的旋轉與扭曲(可微的幾何變換)，最後拼湊出漂亮的圖案（正確答案）

成功關鍵

- 足夠的數據、超快的電腦（GPU)、以及像 Keras/TensorFlow 這樣好用的工具



現狀：
AI 的侷限性



核心概念

- 現在的 AI 其實是在「背答案」而非「理解」
- 很脆弱：只要在圖片加一點點干擾，AI 可能就會把熊貓看成長臂猿
- 沒常識：它不知道現實世界的運作邏輯（例如地心引力）

成功關鍵

- 足夠的數據、超快的電腦（GPU）、以及像 Keras/TensorFlow 這樣好用的工具

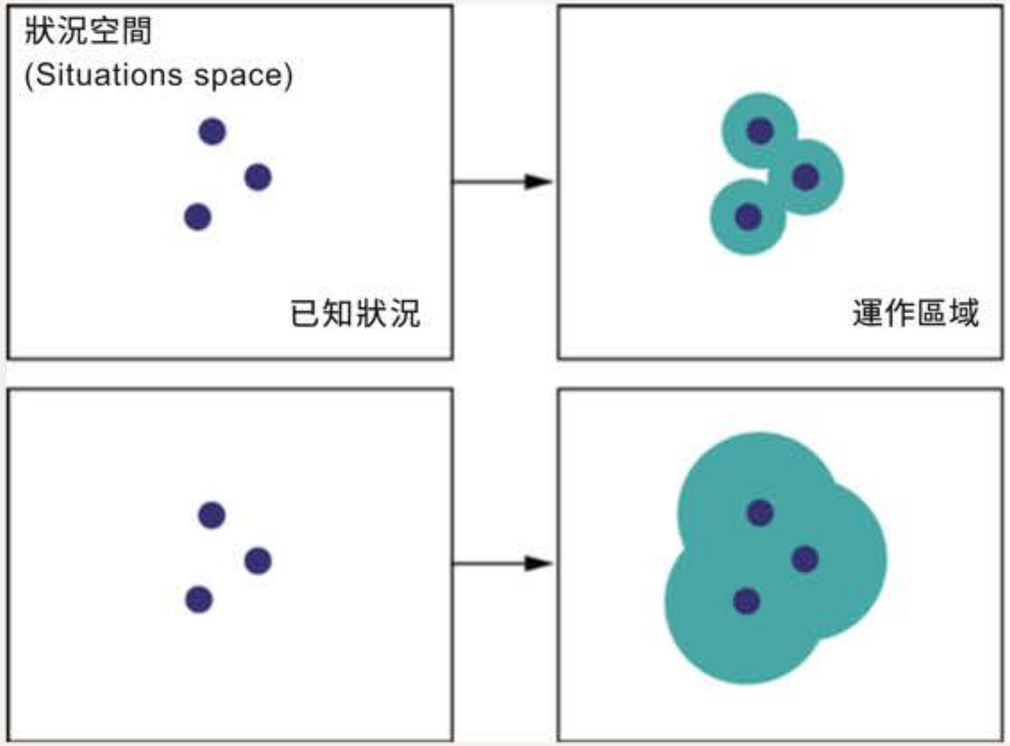
目標:
讓AI學會舉一反三

核心概念

- 目前的 AI 需要吃海量數據才能學會一件事
- 但對人類來說可能只需要看一兩次

目標

- 從局部泛化（只會做練過的題目）進步到極限泛化（沒看過的題目也能靠邏輯推出來）



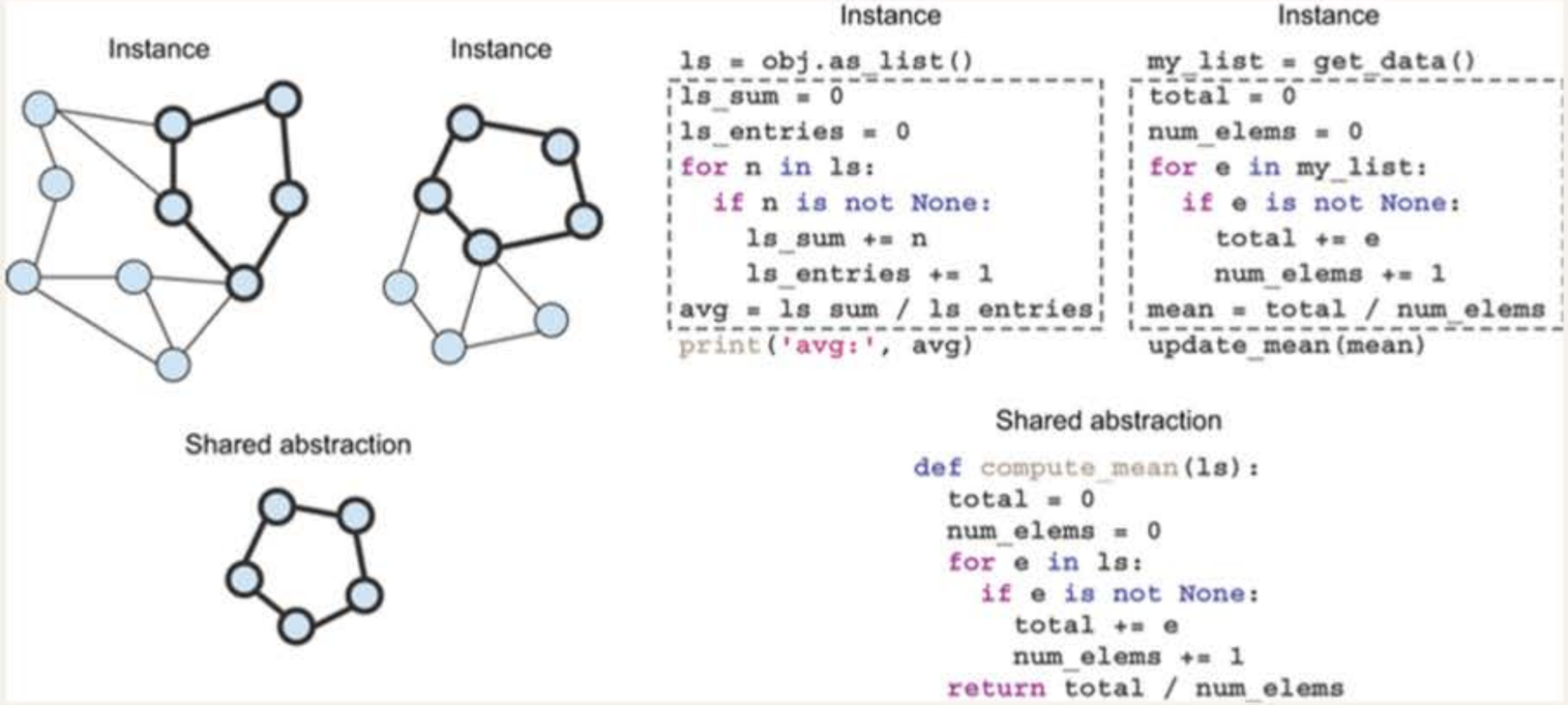
低智能系統:
資訊轉換率較低

高智能系統:
資訊轉換率較高

關鍵：
加入邏輯與搜尋

核心概念

- 現在的 AI 靠直覺（像反射動作），未來的 AI 需要靠寫程式（像思考步驟）
- 模組化：把複雜問題拆解成一個個小零件，再重新組裝



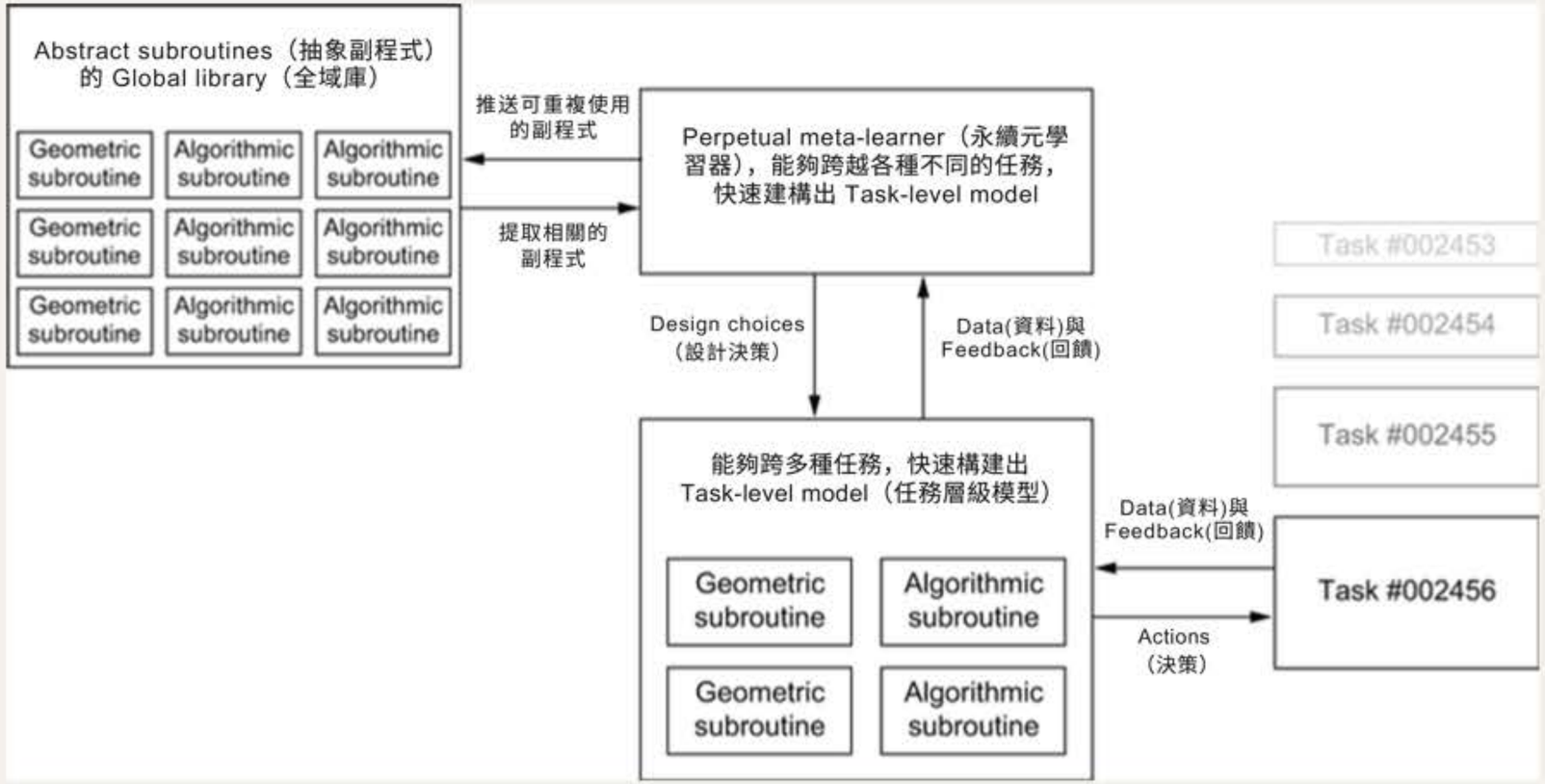
透過辨識不同程式碼片段（Instance），將其提取並封裝成一個通用的函式（Shared abstraction），藉此減少重複代碼並提升程式的可重用性與簡潔度。

AI 應該學會提取通用的規則，而不是死記硬背

未來：
自動進化的系統

核心概念

未來的 AI 是一台會升級的機器。它每解決一個問題，就會把學到的方法存進圖書館，下次遇到更難的問題時，直接去圖書館借書來參考



這是一個 Perpetual Meta-Learner(永續元學習器)的進化循環：它從 Global library(全域庫)中 Fetch(提取)抽象子程式來動態組裝 Modular task-level program(模組化任務程式)，並在解決任務後將新知識 Push(推回)庫中，實現知識的持續累積與快速遷移。

11246062 邱偉豪 、11246064 張庭語、11246084 鍾百陶、11246085 張家瑜
資管三乙

Thank you

專題學習進度檢核報告 - 第二週

指導教授： 蒯思齊

2026/01/27
WEEK2